Módulo 5

Variables y Tipado

Curso: TypeScript

60 min

Objetivos de Aprendizaje

- Dominar la declaración de variables con **let, const y var**
- Comprender la **inferencia de tipos** de TypeScript
- Aplicar **type annotations** de manera efectiva
- Utilizar **tipos personalizados** y aliases
- Implementar **type assertions** de forma segura

1. Contexto

Declaraciones de Variables

let, const y var

TypeScript hereda las declaraciones de variables de JavaScript con verificación de tipos:

```
// const - valor que no cambia (recomendado)
const API_URL = "https://api.example.com";
const MAX_RETRIES = 3;

// let - valor que puede cambiar
let currentUser: string = "Juan";
currentUser = "María"; // OK

// var - evítalo (function-scoped, no block-scoped)
var oldStyle = "deprecated"; // No recomendado
```

Best Practice: Usa `const` por defecto, usa `let` solo cuando necesites reasignar.

2. Inferencia de Tipos

Inferencia de Tipos

TypeScript infiere el tipo automáticamente cuando no lo declaras explícitamente:

```
// TypeScript infiere: string
let name = "Juan";

// TypeScript infiere: number
let age = 30;

// TypeScript infiere: string[]
let tags = ["urgent", "work"];

// TypeScript infiere: { name: string, age: number }
let user = { name: "Ana", age: 25 };
```

Lab: Aplicar Tipado al Sistema de Tareas

Objetivo: Definir tipos personalizados y aplicar type annotations al Caso 1.

1. Abre el archivo `src/types/index.ts`
2. Crea un type alias para `TaskPriority`
3. Crea un type alias para `TaskStatus`
4. Define la interfaz `Task` con tipos explícitos
5. Usa type assertion para convertir datos sin tipo

Código del Lab

```typescript
// 1. Type aliases
type TaskPriority = "low" | "medium" | "high";
type TaskStatus = "pending" | "inProgress" | "completed";

// 2. Interfaz Task
interface Task {
  id: string;
  title: string;
  description: string;
  priority: TaskPriority;
  status: TaskStatus;
  assigneeId?: string;
  completedAt?: Date;
}

// 3. Type assertion
const rawData = { id: "1", title: "Nueva tarea" };
const task: Task = rawData as Task;
```

3. Type Aliases y Unions

Tipos Personalizados

Los type aliases permiten crear nombres para tipos complejos:

```
// Union de tipos
type ID = string | number;

type Priority = "low" | "medium" | "high";

type Status = "active" | "inactive" | "pending";

// Tipo con propiedades
type User = {
  id: ID;
  name: string;
  email: string;
};

// Intersection de tipos
type Admin = User & { role: "admin" };
```

Lab: Tipos Avanzados para el Sistema de Tareas

Objetivo: Crear tipos personalizados que combinen union y intersection.

1. Crea un tipo `SerializableTask` que convierta fechas a strings
2. Crea un tipo `UrgentTask` con urgencia
3. Crea un tipo `TaskFilter` para opciones de filtrado

Código del Lab

```
// SerializableTask
type SerializableTask = Omit<Task, "completedAt"> & {
  completedAt?: string;
};

// UrgentTask
type UrgentTask = Task & {
  isUrgent: true;
  urgencyReason: string;
};

// TaskFilter
type TaskFilter = {
  status?: TaskStatus;
  priority?: TaskPriority;
  assigneeId?: string;
};
```

Resumen

Puntos Clave

- Usa `const` por defecto, `let` solo cuando sea necesario
- TypeScript infiere tipos automáticamente en la mayoría de casos
- Usa annotation explícita para parámetros y retornos de funciones
- Los type aliases crean nombres reutilizables para tipos complejos
- `as` assertions permiten convertir tipos (úsalo con precaución)

Ejercicio Integrador

Sistema de Biblioteca

Caso 2: Biblioteca

Objetivo: Extender el Caso 2 con tipos personalizados.

1. Crea tipos para `MemberType` (standard, premium, student)
2. Crea un tipo `LoanFilter` con filtros opcionales
3. Crea un tipo `BookStats` que combine datos de préstamos

Trabajo en grupos de 2-4 personas.

Tips y Gotchas

✅ Tips

- Usa `const` por defecto; solo `let` si necesitas reasignar
- Type inference funciona en el 90% de los casos
- Annotation explícita: parámetros, retornos, variables antes de inicialización

⚠️ Gotchas

- `const` no hace inmutables los objetos - usa `readonly` o `as const`
- Type assertion (`as`) NO valida tipos en runtime
- `type` vs `interface` - type es más flexible para unions

Referencias

Recursos Adicionales

- TypeScript Type Aliases
- TypeScript Inference
- TypeScript Utility Types

Gracias!

Módulo 5 Completado

Siguiente: Módulo 6 - Estructuras de Control